Predictive and Slice-Aware Load Balancing and Auto-Scaling of AMFs in the 5G Core Control Plane

Jaivant Kosaraju (231IT028), Paluru Pavan Kumar (231IT046) , Paluvadi Dinesh Manideep (231IT047)
Department of Information Technology
National Institute of Technology Karnataka, Surathkal, India

***Abstract*—The access and mobility management function (AMF) is the entry point of the 5G core control plane, and its latency directly affects gNB association and UE registration. ScaleLB, a recent load-balancing and auto-scaling approach, places a round-robin balancer in front of several AMFs and adds AMFs when a utilisation threshold is crossed. Its own experiments show that gNB association latency rises sharply while a newly created AMF, which takes roughly 40 s to become functional, is still starting. We propose two extensions to ScaleLB: (i) slice-aware priority queuing that shields URLLC signalling from mMTC bursts, and (ii) an LSTM-based predictive auto-scaler that boots AMFs ahead of forecast demand. We describe a hybrid evaluation plan that combines a real Open5GS testbed with a discrete-event simulator, and we state our expectations as hypotheses to be tested rather than as results. This report covers the problem, the design and the methodology.**

***Index Terms*—5G core, control plane, AMF, load balancing, network slicing, auto-scaling, LSTM, traffic forecasting**

I. Introduction

The control plane is one of the main sources of signalling overhead in mobile core networks, and the growth of IoT deployments with very large numbers of devices makes the problem harder. The service-based architecture of the standalone (SA) 5G core allows network functions (NFs) to run as virtual machines or containers, so that their number can follow demand [5]. Within the core, the AMF terminates the N2 interface towards the gNBs and handles registration and authentication, which makes it the critical entity for control-plane latency.

Dev et al. proposed ScaleLB [1], which places a load-balancing and auto-scaling module between the RAN and a pool of AMFs. The module hides the core from the RAN, spreads control messages over the AMFs with round-robin, and scales the number of AMFs with either a proactive threshold rule or a reactive rule. The authors implemented it in Open5GS and reported that it achieves latency competitive with a statically provisioned pool while using fewer active AMFs. They also observed latency spikes whenever the existing AMFs were exhausted, because a new AMF needs about 40 s to start, and they named machine-learning-based prediction as future work.

This project builds on ScaleLB and addresses two limitations visible in that paper: scaling decisions are not predictive, and no traffic class is protected while AMFs are starting. The contributions we plan are:

- a *slice-aware priority queue* that dispatches URLLC requests without queuing and rate-limits mMTC requests under high AMF load;
- an *LSTM-based predictive auto-scaler* whose forecast horizon exceeds the AMF start-up time;
- an evaluation on an Open5GS testbed and a SimPy simulator, comparing the proposed design with the ScaleLB policies.

The rest of the report is organised as follows. Section II reviews background and related work, Section III states the problem and objectives, Section IV presents the architecture, and Section V describes the methodology.

II. Background and Related Work

A. AMF and the 5G Control Plane

In 5G SA, a gNB first sets up an NGAP association with an AMF over SCTP (N2 interface). A UE then registers, is authenticated, and finally requests a PDU session.

B. ScaleLB

ScaleLB [1] considers B active gNBs, A active AMFs and a capacity of α gNBs per AMF. In the proactive variant, a new AMF is started when

$$\frac{B}{A \times \alpha} \geq \gamma, \tag{1}$$

where γ is a predefined utilisation threshold. In the reactive variant, a new AMF is started when a new association request makes

$$\frac{B}{A \times \alpha} > 1. \tag{2}$$

Messages are mapped to AMFs by round-robin. The prototype uses Open5GS, my5G-RANTester and LoxiLB on Kubernetes. The evaluation uses 90 UE–gNB pairs with uniform random arrivals, $\alpha = 30$, a maximum of three AMFs and $\gamma \in \{0.7, 0.8, 0.9, 1.0\}$, and compares both variants with a static pool. The paper reports spikes in association latency whenever a new AMF is needed and attributes them to its roughly 40 s start-up time; requests are queued until it is ready. The scheme only scales up.

C. Related Work

Nguyen et al. [2] proposed dynamic control-plane load balancing driven by AMF load, service time and the number of pending requests, and showed gains over static policies such as round-robin and consistent hashing. Bello et al. [3] proposed predictive container auto-scaling for the 4G evolved packet core based on CPU utilisation of the network functions. LSTM networks [4] are a standard tool for sequence forecasting. We do not claim novelty of the individual techniques, only their combination and measurement on top of ScaleLB in this setting.

III. Problem Statement and Objectives

A. Problem Statement

Given a pool of AMFs behind a load balancer whose size changes at run time, and a start-up delay T_{boot} for each new AMF, find queuing and scaling policies that (a) keep control-plane latency low for latency-critical slices and (b) avoid queuing caused by T_{boot}, while limiting the number of AMF-seconds consumed.

B. Objectives

- **O1 Slice-aware priority queuing.** Protect URLLC signalling from mMTC bursts. *Hypothesis:* URLLC tail latency stays far below mMTC latency under an IoT storm, at the cost of extra mMTC queuing delay.
- **O2 LSTM-based predictive auto-scaling.** Forecast the arrival rate at least T_{boot} ahead and boot AMFs early. *Hypothesis:* on predictable traffic, queuing during AMF start-up is reduced relative to threshold scaling, with a measurable over-provisioning cost.

Numerical targets are deliberately not fixed in advance; they will be determined by the experiments.

IV. Proposed System Architecture

Fig. 1 shows the proposed design. The ScaleLB module keeps its position between the RAN and the AMFs and keeps the core hidden from gNBs. Inside it, a slice classifier reads the slice identifier (S-NSSAI) of each request, priority queues and a rate limiter control admission, and a dispatcher sends each admitted request to the least-loaded active AMF. A monitor feeds an LSTM forecaster, whose output drives the auto-scaler that starts or drains AMFs.

V. Methodology

A. System Model and Assumptions

Entities. UEs belong to a slice (URLLC, eMBB or mMTC) and generate control-plane requests such as registration, authentication and PDU session setup. The load balancer classifies requests, queues them, selects an AMF and runs the scaler. Each AMF has a service rate μ (requests/s), and a state (Booting, Active or Draining).

Granularity. ScaleLB balances at gNB level. For the slice-aware queue we work with individual control-plane requests, each tagged with its slice; the baseline comparisons use gNB-level arrivals as in the paper.

Assumed parameters. μ and the slice mix are not given by ScaleLB; they are set from measurements on the testbed where possible and otherwise swept in sensitivity studies.

B. Comparison Baselines

We compare against the ScaleLB setting: 90 gNBs with uniform random arrivals, $\alpha = 30$, at most three AMFs, $T_{\text{boot}} \approx 40$ s and $\gamma \in \{0.7, 0.8, 0.9, 1.0\}$. The policies are Static (all AMFs on), React (Eq. 2) and Pro (Eq. 1), each with round-robin dispatch and a single queue. We add one baseline, WarmStandby, which keeps one pre-booted spare AMF. It removes the start-up delay without any prediction, so it is the baseline the forecaster must beat on cost.

C. Workload Generation

Uniform random arrivals cannot be predicted by any model, so besides the paper-like workload we generate structured traces: (1) diurnal, a sinusoid with noise; (2) ramp, a steady increase; (3) flash crowd or mMTC storm, a sudden burst of IoT registrations; and (4) Markov-modulated Poisson bursts. The slice mix is initially 5% URLLC, 35% eMBB and 60% mMTC and is varied in a sensitivity study. For the paper-like workload we expect prediction to help little, and we will report this.

D. Slice-Aware Priority Queuing (O1)

Fig. 2 shows the flow. Each request carries a slice identifier that the balancer reads before dispatch. URLLC requests bypass the queue and are dispatched immediately to the least-loaded active AMF. eMBB requests use a normal FIFO queue and are then sent to the least-loaded AMF. mMTC requests are held in a buffered queue, and when the mean AMF CPU utilisation exceeds 80% their release is limited by a token bucket. To avoid starvation, mMTC requests are aged so that their maximum waiting time is bounded, and a fraction (e.g. 10–15%) of AMF capacity is reserved as headroom for URLLC. We report per-slice latency percentiles, mMTC queuing delay and maximum wait.

E. LSTM-Based Predictive Auto-Scaling (O2)

Forecasting task. Request counts are aggregated in 1 s bins. The model input at time t is the last $W = 60$ s of the arrival rate, plus simple features (recent slope, rolling mean and standard deviation, and time of day when the trace has one). The target is the arrival rate $\lambda(t + H)$ with

$$H \geq T_{\text{boot}} + \Delta, \tag{3}$$

where Δ is the decision interval; with $T_{\text{boot}} \approx 40$ s we use H between 45 and 60 s. A shorter horizon cannot hide the start-up delay.

Models. We compare, in order of complexity: an EWMA or moving average, linear regression or ARIMA, a Random Forest regressor, and a small LSTM [4] (one or two layers of 32–64 units, MSE loss, Adam optimiser). The simplest model that performs adequately is preferred. Traces are split by time (training, validation, test) without shuffling across time. Forecast quality is measured by MAPE and RMSE.

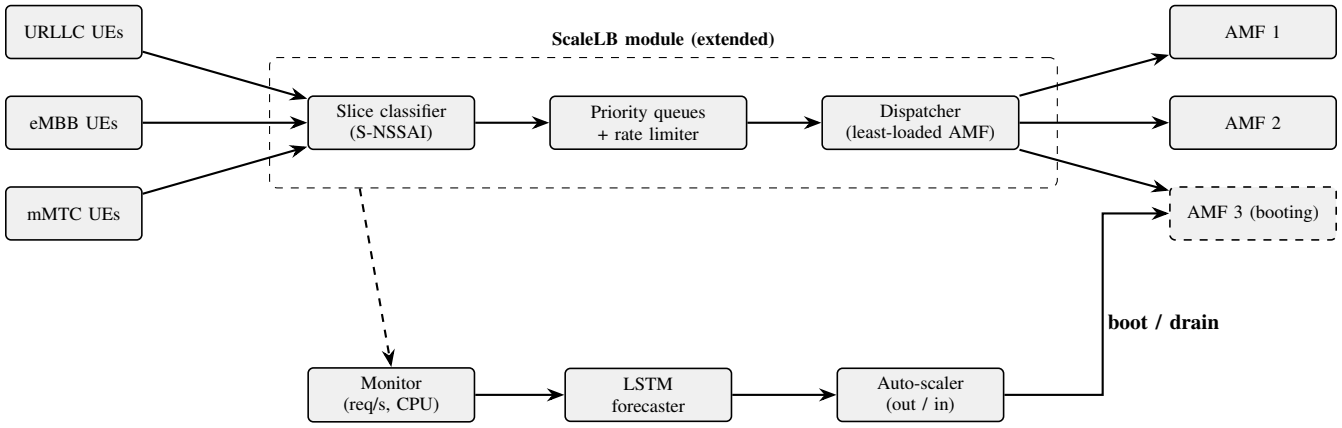


Fig. 1. Proposed architecture. Slice classification, priority queuing and least-loaded dispatch extend the ScaleLB module; the LSTM forecaster drives the auto-scaler that starts or drains AMFs.

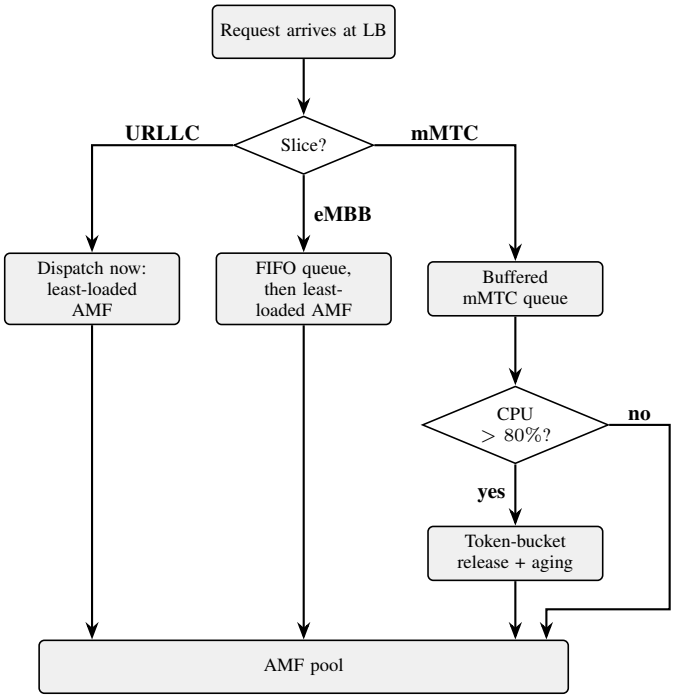


Fig. 2. Methodology for O1: slice-aware queuing and dispatch. URLLC bypasses queuing; mMTC is buffered and throttled when AMF CPU is high.

Scaling rule. With forecast $\hat{\lambda}(t+H)$, service rate μ, target utilisation γ and safety margin m, the required number of AMFs is

$$N(t) = \left\lceil \frac{\hat{\lambda}(t+H)\,(1+m)}{\mu\,\gamma} \right\rceil . \tag{4}$$

If $N(t)$ exceeds the number of active plus booting AMFs, the difference is started immediately (Fig. 3). *Scale-in* happens only if the forecast stays below a lower threshold (e.g. 0.4 utilisation) for a sustained period; the AMF is then marked DRAINING, accepts no new sessions and stops after its current sessions finish. This hysteresis prevents oscillation.

Failure analysis. We inject deliberately wrong forecasts (over- and under-prediction) and report the resulting drops, latency and AMF-seconds, because the benefit holds only when the forecast is accurate. We expect little gain on the paper-like uniform arrivals.

F. Implementation Platforms

Real 5G testbed. We deploy Open5GS with two or three AMFs registered to a shared NRF, an emulated RAN (UERANSIM or my5G-RANTester) and LoxiLB or our own proxy as the load balancer, on Docker or Kubernetes (SCTP support is required). The testbed is used to run the ScaleLB baseline policies, to measure the real AMF start-up time and per-registration service time for calibrating the simulator, and to run the predictive controller, a Python process that reads request counts and scales the AMF deployment through the Kubernetes API.

Discrete-event simulator. A SimPy model contains UEs, AMFs (service rate, boot delay) and the balancer with pluggable queuing and scaling policies. Every request is logged with arrival time, slice, chosen AMF, queue wait, service time and drop status, and all metrics derive from this log. We run at least ten random seeds per configuration and report means with confidence intervals. The simulator allows thousands of UEs and 5–10 AMFs, which a single-machine testbed cannot support.

G. Metrics and Experimental Plan

We report: per-slice latency (p50, p95, p99) and the instantaneous association latency used in the base paper; requests queued or dropped during AMF start-up; per-AMF load; AMF-seconds and mean utilisation as cost; forecast error (MAPE, RMSE); and the added delay and maximum wait of mMTC traffic. Table I lists the planned experiments. Results will be reported in the next stage of the project.

H. Limitations

The simulator is not a real core, so its parameters are justified with the base paper's numbers and measurements

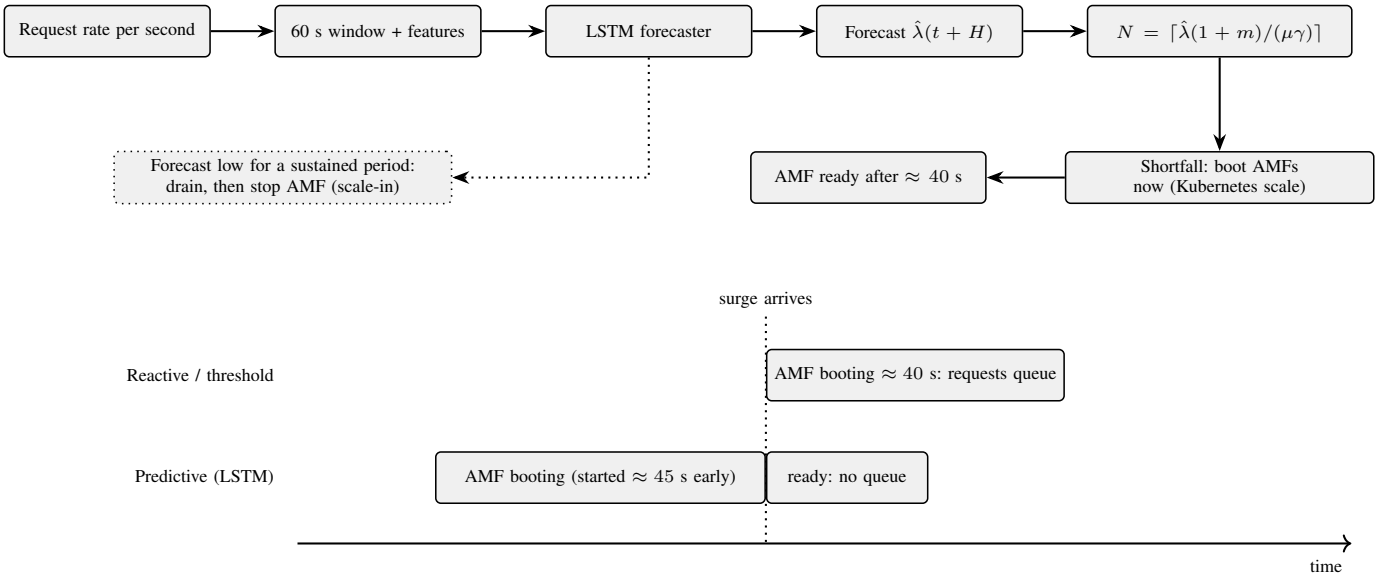


Fig. 3. Methodology for O2: forecast, decide, boot early. The lower panel contrasts a reactive scaler, whose AMF starts when the surge arrives, with a predictive scaler whose horizon exceeds T_{boot}.

TABLE I
PLANNED EXPERIMENTS

ID	Experiment	Compared / purpose
E1	Forecast accuracy	EWMA, linear, Random Forest, LSTM
E2	Ramp and diurnal load	React, Pro, warm standby, predictive
E3	Flash crowd (mMTC storm)	Priority queue on and off
E4	Scale-out and scale-in	Cost and stability with hysteresis
E5	Forecast error injection	Cost of wrong forecasts
E6	Component study	Queue and predictor switched on one at a time
E7	Sensitivity	γ, H, T_{boot}, slice mix

from the testbed and are swept for sensitivity. Prediction can only help on predictable traffic. Assumed parameters (μ, slice mix) are not taken from ScaleLB. If time is limited, the LSTM is the first component to be reduced in scope, and the scaler would use the Random Forest or linear model instead.

REFERENCES

[1] W. Dev, S. Bera, and A. Tiwari, "Control-plane load balancing and auto-scaling in 5G and beyond networks," in *Proc. Int. Conf. Information Networking (ICOIN)*, 2025, pp. 83–87.

[2] V.-G. Nguyen, K.-J. Grinnemo, J. Taheri, and A. Brunstrom, "Adaptive and latency-aware load balancing for control plane traffic in the 4G/5G core," in *Proc. Joint European Conf. Networks and Communications & 6G Summit*, Jun. 2021, pp. 365–370.

[3] Y. Bello, A. A. Abdellatif, M. S. Allahham, A. R. Hussein, A. Erbad, A. Mohamed, and M. Guizani, "B5G: Predictive container auto-scaling for cellular evolved packet core," *IEEE Access*, vol. 9, pp. 158204–158214, 2021.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] 3GPP, "System architecture for the 5G System (5GS)," TS 23.501.

[6] Open5GS project. [Online]. Available: https://open5gs.org/